

Overall Instructions

Please read the following instructions carefully before starting.

Structure of the task

The task is divided into a mandatory Layer 1 and optional layers. Layer 1 is the only part we require. The optional layers exist for candidates who want to go further, but attempting them is not expected and will not compensate for a weak Layer 1. A solid, well-understood Layer 1 is worth more to us than a quick attempt at all layers.

Use of AI

We encourage you to use AI tools if you wish. We use them ourselves in our company and consider them part of the modern engineering toolkit. What matters is that you understand and can defend the key decisions in your submission.

What we evaluate

We are not looking for the largest scope. We are looking for depth of understanding. Concretely:

- Every architectural choice should be defended: what you chose, what the alternatives were, and why you made that choice.
- We will discuss your submission in a follow-up conversation. Expect questions on any decision in your code or documentation.

Assignment: Confidential ML Model Distribution PoC

Summary

Design and implement a proof-of-concept for a confidential LLM model delivery pipeline. The task has three progressive layers: Layer 1 is required and Layer 2 and 3 are optional. All layers share the same two-part structure: a Producer that encrypts and publishes a model, and a Consumer that retrieves and decrypts it. No confidential hardware is required at any layer.

Layer 1 (**Required**): *Encrypted Model Distribution in Kubernetes*

Implement a basic confidential model delivery pipeline running in a Kubernetes environment.

Producer

1. Select a small open model from Hugging Face (e.g. a small BERT).
2. Encrypt the model (free to choose any tool/library).
3. Push the encrypted artifact to Hugging Face Hub.
4. Store the decryption key as a Kubernetes Secret, accessible to the consumer workload.

Consumer

1. Deploy a Kubernetes pod that mounts the decryption key from the Kubernetes Secret.
2. Download the encrypted artifact from Hugging Face Hub.
3. Decrypt the model using the key.
4. Load the model.

Layer 2 (**Optional**): *Model Signing & Verification*

Extend Layer 1 by adding a signing step to the producer and a verification step to the consumer, ensuring the model artifact was not tampered with between publication and use.

Additional steps on top of Layer 1:

1. Generate an asymmetric key pair (free to choose any tool/library).
2. Sign the encrypted artifact with the private key during the producer phase.
3. Publish the signature alongside the artifact.
4. In the consumer, verify the signature using the public key before decryption takes place (abort if verification fails).

Layer 3 (**Optional**): *Attested Key Release via Kata+CoCo*

Extend the pipeline by replacing the Kubernetes Secret with attested key retrieval using Confidential Containers in development mode. The consumer no longer trusts the host for key delivery, so the decryption key is only released after an attestation. This layer can be

implemented on top of Layer 1 alone or combined with Layer 2 for the full confidential pipeline (encrypt + sign + attest).

Additional steps on top of Layer 1:

1. Deploy the CoCo operator and Trustee KBS on the cluster using the “kata-qemu-coco-dev” runtime class.
2. Store the decryption key in KBS via “kbs-client set-resource” under a defined resource path (e.g. default/key/my-model) instead of a Kubernetes Secret.
3. Configure a permissive resource policy in KBS allowing key release to sample TEE attestation.
4. Run the consumer pod with the KBS address configured via the “agent.aa_kbc_params” parameter annotation.
5. Fetch the decryption key from KBS through the Confidential Data Hub (CDH) endpoint at 127.0.0.1:8006/cdh/resource/..., relying on the full attestation flow.

General Notes:

- Layers 2 and 3 are independent: a candidate may implement Layer 1 + Layer 3 (attested delivery without signing) or Layer 1 + Layer 2 + Layer 3 (the full stack).
- The choice of model, encryption library, and tooling is left to the candidate (justification might be asked during the discussion of the solution).

Deliverables

A public Git repository containing:

- Dockerfiles for the producer and consumer workloads.
- Kubernetes manifests for all workloads (Pods, Secrets, or any other resources used).
- A README covering prerequisites, how to build and deploy the full pipeline, and how to verify each layer works.

Reference: [Confidential Containers without confidential hardware](#)